

Livrable 2 : MongoDB

FAR Adam INFO FISE 4A

INTRODUCTION

L'objectif de cette phase 2 était de migrer les données de cinéma d'un modèle relationnel (SQLite) vers un modèle orienté documents (MongoDB). Cette transition vise à améliorer les performances de lecture pour une application de consultation de films à l'aide de la technique de dénormalisation.

II) Migration

Le choix s'est porté sur MongoDB pour sa flexibilité de schéma et sa capacité à gérer la dénormalisation. Contrairement à SQLite, MongoDB permet de stocker des objets complexes (listes, sous-documents) au sein d'une seule unité de stockage. La migration s'est faite en deux étapes distinctes qui ont été mises en œuvre afin de garantir l'intégrité des performances. La création de documents plats et de documents structurés.

i) Phase Plate

Les données ont d'abord été transférées de SQLite vers MongoDB en conservant la structure originale. Chaque table SQL est devenue une collection MongoDB (movies, persons, ratings, genres, etc...). Pour cela, on a utilisé des outils dans les scripts Python comme sqlite3 et pymongo.

ii) Phase Structurée

Pour exploiter au mieux la puissance du NoSQL, nous avons généré une nouvelle collection dénormalisée nommée movies_complete via un pipeline d'agrégation. Cette version est améliorée par rapport à la version plate car chaque film contient maintenant ses caractéristiques sous forme

de liste et directement dans le même document. Cela permet de supprimer des jointures (\$lookup) lors des requêtes fréquentes sur les films, ce qui rend ces opérations moins coûteuses. En clair, elle fusionne les données essentielles en un document unique. Le document est de la forme suivante :

```
{
  _id: ObjectId('694f393bba88d5189d4a277f'),
  movie_id: 'tt0002130',
  titleType: 'movie',
  primaryTitle: "Dante's Inferno",
  originalTitle: "L'Inferno",
  isAdult: 0,
  startYear: 1911,
  endYear: null,
  runtimeMinutes: 71
}
```

III) Processus de migration et d'indexation

La migration a été réalisée grâce à l'automatisation d'un script Python. Le processus comprend 3 étapes. Tout d'abord, on effectue le nettoyage des données. Ensuite, on effectue une agrégation sur les tables comme genres et ratings vers les films. Enfin, on crée la collection finale grâce à \$out.

Concernant la stratégie d'indexation, on a déployé des index comme les index multiclés sur le champ genres pour filtrer instantanément par catégorie, les index composés pour les tris par année et par note ou bien les index textuels pour la recherche floue sur les noms d'acteurs.

IV) Implémentation des requêtes mongoDB équivalentes

La requête Q1 illustre la transition vers un modèle hybride où l'obtention d'une filmographie nécessite un pipeline à trois niveaux (\$match, suivi de deux \$lookup). Cette méthode de jointure à la volée démontre que

MongoDB, bien que flexible, exige une indexation rigoureuse pour égaler la vitesse du SQL sur des données volatiles. Pour la Q3, la complexité réside dans le traitement des tableaux via l'opérateur \$unwind, qui déconstruit les listes de rôles avant d'appliquer un regroupement par couple acteur/film et un filtrage, permettant ainsi d'identifier les performances multiples au sein d'une même œuvre.

Concernant la Q7, l'introduction de \$setWindowFields marque une avancée majeure en reproduisant les fonctions de fenêtrage SQL (PARTITION BY). En partitionnant par genre et en appliquant un \$denseRank, le calcul est directement délégué au serveur, ce qui minimise le transfert réseau en ne renvoyant que les résultats finaux. Parallèlement, les agrégations de groupe comme la Q5 et la Q8 profitent de la dénormalisation et des accumulateurs conditionnels (\$cond) pour traiter des millions de documents en une seule fois, surpassant parfois la longueur des sous-requêtes SQL.

V) Complexité

Si le SQL est plus compact (quelques lignes de texte), le Framework d'Agrégation de MongoDB est plus modulaire. Chaque étape du pipeline est un bloc JSON indépendant. Cette structure facilite le débogage : on peut isoler chaque étape pour inspecter la transformation des données avant de passer à la suivante. Cela rend la maintenance du code plus robuste pour des applications à grande échelle.

VI) Résultats

Requête	SQLite (ms)	MongoDB (ms)	Ratio
Q1 Filmographie	2429.70	1082.57	x0.4
Q2 Top N Genre	24.99	32.50	x1.3
Q3 Multi-roles	940.35	77141.79	x82.0
Q4 Collaborations	904.44	1018.38	x1.1
Q5 Popular Genres	56.94	70.45	x1.2
Q6 Career Decades	2235.65	1037.54	x0.5
Q7 Rank Genres	162.01	396.44	x2.4
Q8 Breakthrough	2573.90	25417.94	x9.9

MongoDB présente des avantages. En effet, les requêtes Q1 (Filmographie) et Q6 (Carrière) montrent un gain de performance de 50 % (Ratios x0.4 et x0.5). Ce résultat valide la stratégie de dénormalisation : en stockant des documents structurés et pré-agrégés, MongoDB réduit les entrées/sorties disque et évite la reconstruction coûteuse des données en mémoire.

En observant les Performance de SQLite, on remarque une anomalie pour Q3 (Multi-rôles) : Le ratio de x82 en faveur de SQLite souligne la difficulté de MongoDB à traiter des opérations de dédoublonnement massif via \$unwind sur des volumes importants sans structure dédiée.

On remarque aussi analytiquement sur les Q7 et Q8 que SQLite surclasse MongoDB sur les fonctions de fenêtrage et les calculs conditionnels. L'optimiseur relationnel de SQLite gère plus efficacement les jointures transversales complexes.

On peut en déduire, sur les requêtes Q2, Q4 et Q5 que les deux systèmes sont presque aussi efficaces l'un que l'autre (Ratios proches de 1). Pour un volume de ~36 000 films, SQLite reste extrêmement léger et performant.

CONCLUSION

Cette conclusion met en lumière la complémentarité stratégique des deux architectures. Le passage au NoSQL s'avère hautement bénéfique pour l'interface utilisateur, car il privilégie la vélocité de consultation. Grâce à la dénormalisation, MongoDB permet une "lecture atomique" : toutes les données nécessaires à l'affichage d'une fiche (genres, notes, acteurs...) sont récupérées en un seul accès disque, garantissant une bonne réactivité.

À l'inverse, le modèle relationnel reste préférable pour les traitements analytiques profonds et la gestion complexe de l'intégrité. SQLite excelle dans l'exécution de calculs transversaux et de statistiques globales grâce à son optimiseur de requêtes, capable de traiter des relations complexes sans duplication de données. Alors que MongoDB favorise la performance de lecture au prix d'une redondance, le SQL garantit une cohérence absolue et une flexibilité analytique supérieure, prouvant que l'architecture idéale pour un projet d'envergure repose souvent sur une coexistence hybride entre ces deux technologies.